

Integración, SOA y Microservicios

Point-to-point → ESB → SOA → microservicios · DORA · Conway · patrones de Richardson

Ingeniería de Software II · ITBA · 2026-1C · Repaso conceptual

RESUMEN

Las formas de integrar sistemas no son modas que se reemplazan: son una **escalera evolutiva** (point-to-point → ESB → SOA → API + microservicios) sobre tres ejes — más *business ownership*, más desacoplamiento, audiencia más amplia. **Subir un escalón no es siempre mejor**: cada nivel agrega desacoplamiento pero también complejidad operativa. La decisión es *en qué escalón parar* según el driver de negocio real, no según la moda (source: integracion-de-sistemas.md).

§1 La escalera evolutiva

El problema de la *integración de sistemas* (EAI) es hacer que aplicaciones distintas —distintas épocas, equipos, datos— trabajen juntas. La clase lo ordena no como historia de modas sino como niveles crecientes de exposición de la funcionalidad de negocio (source: integracion-de-sistemas.md).

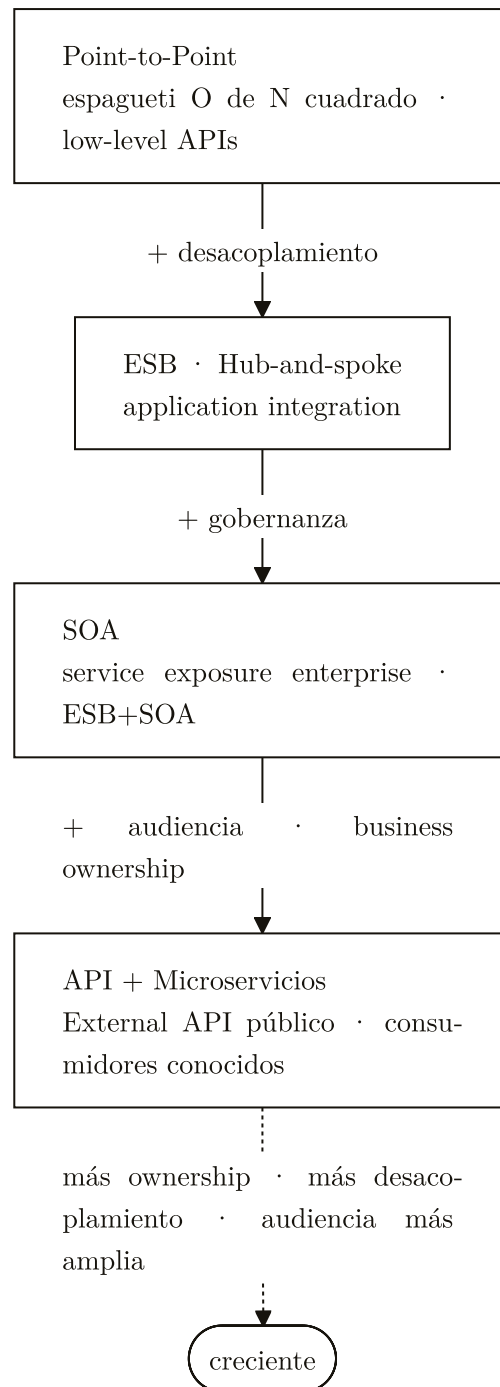


FIGURA 1. La escalera evolutiva de integración: cada escalón suma desacoplamiento, ownership y audiencia, pero también complejidad operativa.

TABLA 1. Los cuatro escalones, en una línea cada uno

Escalón	Mecanismo	Problema que resuelve
Point-to-Point	Cada app habla directo con cada otra; código de integración disperso	(es el problema) Acoplamiento $O(N^2)$, mantenimiento imposible
ESB / Hub	Bus central; cada sistema conecta vía Adapter al bus	Espagueti $\rightarrow$ N conexiones en vez de N^2 . Centraliza transformación/ruteo
SOA	Servicios reutilizables sobre el ESB + Registry + Governance	Integración corporativa gobernada de funcionalidad de negocio
API + Microservicios	Servicios autónomos, comunicación liviana, APIs públicas	Velocidad de cambio, escala independiente, audiencia abierta

DEFINICIÓN

Point-to-Point (90s): cada aplicación se conecta directo con cada otra (ERP $\leftrightarrow$ CRM $\leftrightarrow$ BI $\leftrightarrow$ Billing...). El grafo crece $O(N^2)$: alto acoplamiento, mantenimiento imposible, responsabilidades poco claras. Es el "plato de espagueti" (source: integracion-de-sistemas.md).

DEFINICIÓN

ESB — Enterprise Service Bus: middleware central *hub-and-spoke*; todos los sistemas conectan al bus vía *Adapters*, no entre sí $\rightarrow$ N conexiones en vez de N^2 . Hace **canonical integration** (data translation, routing, composition) y **adaptation** (protocol conversion, transport). Alrededor: BPM/BAM, Governance, Service Registry (source: esb-enterprise-service-bus.md).

CUÁNDO NO • CUIDADO

Límite del ESB: es un punto central $\rightarrow$ cuello de botella, single point of failure y "monolito de integración" donde toda la lógica termina concentrada. El anti-pattern es el "ESB inteligente" con reglas de negocio adentro. Microservicios responde descentralizando: **smart endpoints, dumb pipes** (source: esb-enterprise-service-bus.md; Fowler).

§2 SOA vs Microservicios — el pivote

FUENTE

Mientras SOA busca resolver **primero integración y luego gobernanza**, microservicios busca ante todo permitir a las organizaciones **crecer e ir más rápido** (source: microservicios.md, slide 17).

No es "SOA mejorado": tiene otra motivación de negocio. El atributo de calidad que optimiza microservicios es la **velocidad de cambio** (time-to-market, deploys frecuentes, escala independiente), no la integración corporativa.

TABLA 2. SOA vs Microservicios

Eje	SOA	Microservicios
Driver	Integración → gobernanza	Velocidad y crecimiento
Comunicación	ESB pesado (smart pipe) central	Liviana, punto a punto / mensajería (smart endpoints)
Gobernanza	Centralizada	Descentralizada, equipos autónomos
Datos	Modelos canónicos compartidos	Database per Service (datos por dominio)
Límite	Escalabilidad, fault tolerance, deploys frecuentes	Complejidad de sistema distribuido

CUÁNDO SÍ • VENTAJAS

SOA — cuándo SÍ: organización con muchas aplicaciones heterogéneas (incluido legacy) que necesitan integrarse y gobernarse de forma centralizada; driver = integración corporativa. La clase es explícita: **SOA sigue siendo hoy el modelo más usado para integrar aplicaciones corporativas** (source: soa-service-oriented-architecture.md).

Granularidad de servicios (SOA): grano fino (base de negocio, ej. "obtener saldo") vs grano grueso (compuesto, agrega validaciones y transaccionalidad, ej. "procesar préstamo"). La composición se hace por orquestación o coreografía (source: soa-service-oriented-architecture.md).

El triángulo: arquitectura + organización + procesos (Conway)

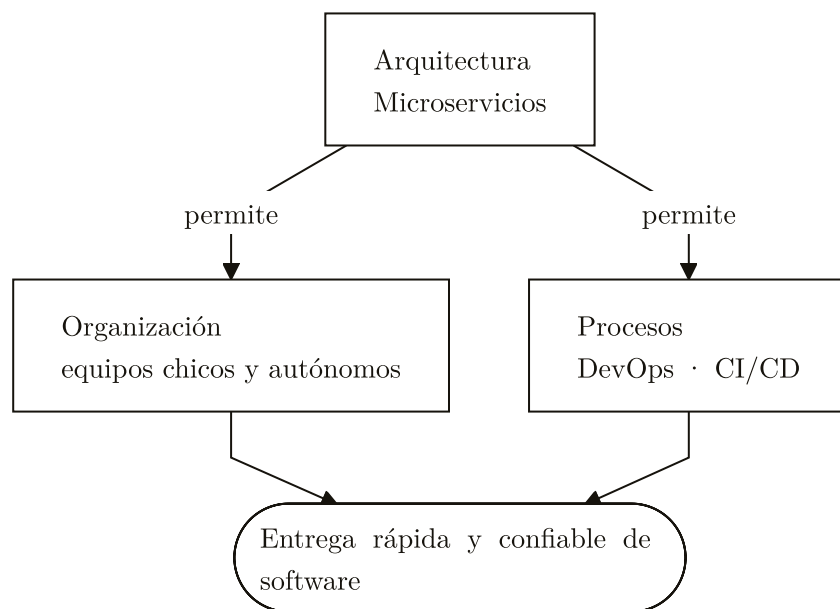


FIGURA 2. Triángulo de Conway: la arquitectura de microservicios habilita equipos autónomos y procesos DevOps/CD, y los tres juntos dan entrega rápida y confiable.

REGLA DE ORO

Ley de Conway en acción: microservicios sin equipos autónomos ni CD no da velocidad — solo da un **monolito distribuido** (lo peor de los dos mundos). La arquitectura habilita los equipos; los equipos necesitan DevOps/CD para no pisarse (source: microservicios.md; Conway 1968).

La justificación cuantitativa: métricas DORA

TABLA 3. High vs low IT performers (2017 State of DevOps, Puppet + DORA)

Métrica	Ventaja high performers
Deployment frequency	46× (on-demand, varios deploys/día)
Lead time for changes	440× (< 1 h; Netflix: 16 min)
Mean Time To Recover (MTTR)	24× (< 1 h)

CUÁNDO NO • CUIDADO

Microservicios — cuándo es over-engineering: en organizaciones chicas o dominios simples es **over-engineering**. La clase: *"se debe decidir prudentemente cuándo usar microservicios, porque existen complejidades de implementación"*. No es gratis — (1) no es fácil definir bien los servicios; (2) múltiples bases → ¿transaccionalidad?; (3) es un sistema distribuido (desarrollo, testeo, deploy más complejos); (4) features que cruzan servicios exigen deploy coordinado. Un monolito hexagonal bien hecho (el FTGO de la clase) es mejor punto de partida. Saltar por moda es anti-pattern (source: microservicios.md; Brooks, "No Silver Bullet").

§3 Catálogo de patrones de Richardson

Microservicios trae "nuevos patrones para nuevos problemas" (Richardson, microservices.io). Se agrupan en **Application/Data**, **Communication** e **Infrastructure**. Por patrón: definición compacta + cuándo aplica / cuándo no.

Descomposición

DEFINICIÓN

Descomposición: definir **dónde cortar** el sistema en servicios — la dificultad central de microservicios. Dos estrategias: **decompose by business capability** (según lo que la empresa *hace*; estable porque las capacidades cambian menos que la tecnología) y **decompose by subdomain** (DDD: cada subdominio → un servicio con su modelo). Ejemplo FTGO: subdominios *Order taking*, *Delivery*, *Kitchen*, *Accounting* → un servicio cada uno (source: [descomposicion-microservicios.md](#)).

CUÁNDO SÍ • VENTAJAS

Buen corte: alta cohesión dentro, bajo acoplamiento entre servicios; alineado con capacidades de negocio o bounded contexts (no con capas técnicas); que **minimice transacciones que cruzan servicios** (esas obligan a sagas); que mapee a equipos autónomos (Conway). Mapa de referencia en banca: **BIAN** (source: [descomposicion-microservicios.md](#)).

CUÁNDO NO • CUIDADO

Mal corte: servicios *chatty* y acoplados → un monolito distribuido. Señales: servicios que siempre se despliegan juntos, transacciones que cruzan todo.

Aggregate (DDD)

DEFINICIÓN

Aggregate: grafo de objetos tratado como **una sola unidad de consistencia**, accedido por su *aggregate root*; puede contener value objects y entidades. Define la **frontera de consistencia transaccional**: una transacción local modifica *un* aggregate atómicamente. Ejemplo FTGO: Order (root) + DeliveryInfo, PaymentInfo, OrderLineItem (source: [aggregate-ddd.md](#)).

*Como cada servicio tiene su propia base, operaciones que cruzan aggregates de servicios distintos **no pueden** ser una transacción ACID única → sagas. Limitaciones: impedance mismatch O-R, no hay historia del aggregate, auditoría tediosa → las ataca Event Sourcing (source: [aggregate-ddd.md](#)).*

Database per Service

DEFINICIÓN

Database per Service: cada servicio es **dueño exclusivo** de su base; nadie más la toca directo. Da autonomía (cada equipo evoluciona su esquema, elige su tecnología → *polyglot persistence*) (source: database-per-service.md).

TRADE-OFF

Shared DB ↔ Database per Service: la *shared database* facilita la transacción única (un commit ACID abarca todo) pero introduce noisy neighbours y acopla esquemas (cambiar una tabla rompe a varios). Separar elige **AVAILABILITY** **SCALABILITY** y autonomía sobre consistencia fuerte → la consistencia entre servicios pasa a ser **eventual** (CAP). Se rompe: ya no hay JOIN cross-servicio (→ API composition / CQRS) ni transacción ACID cross-servicio (→ saga) (source: database-per-service.md).

No confundir con **sharding** (partir una base por volumen) ni **replicación** (copias de la misma base): acá son bases distintas con datos distintos por dominio.

Saga (+ compensaciones)

DEFINICIÓN

Saga: secuencia de transacciones locales. Cada una hace commit ACID local en su servicio y publica un **evento** que dispara la siguiente. Si una falla, se ejecutan **transacciones de compensación** que deshacen los pasos previos *en orden inverso*, sin locks distribuidos. Reemplaza al 2PC (bloqueante, no escala) (source: patron-saga.md).

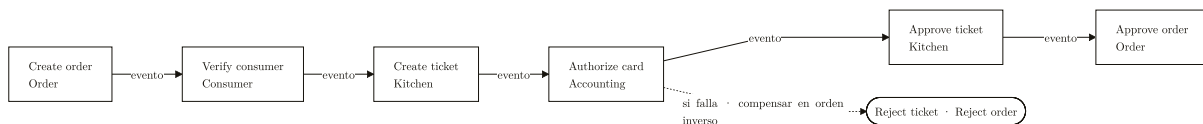


FIGURA 3. Saga FTGO crear pedido: secuencia de transacciones locales encadenadas por eventos; si falla Authorize card se compensa deshaciendo en orden inverso.

TABLA 4. Saga vs 2PC

	2PC	Saga
Atomicidad	Verdadera (todo o nada)	Eventual (con compensaciones)
Locks distribuidos	Sí (bloqueante)	No
Escala / disponibilidad	Baja	Alta
Consistencia	Fuerte	Eventual (CAP)
Complejidad	En el protocolo	En diseñar compensaciones

CUÁNDO NO • CUIDADO

El punto fino: no todo se "deshace" limpio (un mail enviado no se desmanda) → hay que diseñar **compensaciones semánticas** y aceptar una ventana donde el sistema está "a medio camino" (estados intermedios visibles). Se coordina por orquestación o coreografía (source: patron-saga.md).

CQRS

DEFINICIÓN

CQRS: separa **comandos** (escrituras) de **queries** (lecturas). En microservicios se materializa con una **"view DB"** read-only optimizada para consultar, que se mantiene **suscribiéndose a los eventos de dominio** de los servicios dueños. Resuelve consultas cross-servicio sin acoplar las escrituras. Ejemplo: *Order History Service* alimentado por Order/Ticket/Delivery/Accounting events (source: cqrs.md).

CUÁNDO NO • CUIDADO

CQRS — **cuándo no:** si las consultas son simples y caben en un solo servicio, es **over-engineering** (base extra + consistencia eventual sin necesidad). Costo: la view-DB va "atrás" de las escrituras (lag). Si el negocio tolera cero lag, CQRS no aplica → repensar la descomposición (source: cqrs.md).

Event Sourcing

DEFINICIÓN

Event Sourcing: en vez del **estado actual** del aggregate, persiste la **secuencia de eventos de dominio**; el estado se reconstruye reproduciéndolos. Tabla EVENTS: Order Created → Approved → Shipped → Delivered. Resuelve las tres limitaciones del aggregate: historia completa, auditoría/logging gratis, independencia entre sistemas (source: event-sourcing.md).

CUÁNDO NO • CUIDADO

Event Sourcing — **cuándo no:** viene *"a un costo mayor de desarrollo"* (versionado de eventos, reconstrucción con snapshots, consultas más difíciles). Si el dominio no necesita historia/auditoría fuerte, un aggregate clásico es más simple. CQRS + Event Sourcing se combinan natural pero son independientes (source: event-sourcing.md).

API Gateway

DEFINICIÓN

API Gateway: **punto de entrada único** entre consumidores (browsers, mobile, terceros) y los microservicios. Rutea requests, hace **composición de APIs**, traduce protocolos (REST afuera / gRPC adentro) y concentra cross-cutting de borde:

AUTHN/AUTHZ rate limiting, caching, métricas, logging (source: api-gateway.md).

CUÁNDO NO • CUIDADO

Gateway — **riesgos**: cuello de botella / single point of failure → necesita HA + Circuit Breaker. Riesgo de convertirse en **mini-monolito** si se le mete lógica de negocio. No confundir con ESB: el ESB media *entre sistemas internos* (smart pipe); el gateway expone *hacia los consumidores* (capa de borde liviana) (source: api-gateway.md).

BFF — Backend for Frontend

DEFINICIÓN

BFF: variante del API Gateway — **un backend de borde por tipo de cliente** (web, mobile, terceros), cada uno con APIs a medida. Resuelve localización, evita **overfetching** (payloads chicos en mobile) y permite versioning por cliente, mantenido por el equipo de ese frontend (source: backend-for-frontend.md).

CUÁNDO NO • CUIDADO

BFF — **cuándo no**: con un solo tipo de cliente o necesidades homogéneas, un API Gateway único alcanza — varios BFFs serían over-engineering. Contra: más componentes; riesgo de duplicar lógica entre BFFs (compartir lo común en servicios downstream) (source: backend-for-frontend.md).

Circuit Breaker

DEFINICIÓN

Circuit Breaker: ante una falla masiva, los retries generan **efecto bola de nieve** (cascading failure) que impide al servicio recuperarse. El cortacircuitos **corta las llamadas a un servicio que falla** para darle aire. Tres estados (source: circuit-breaker.md):

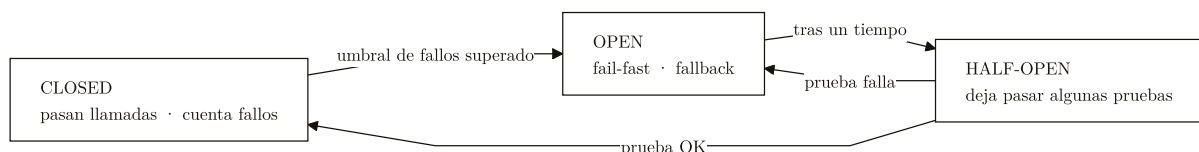


FIGURA 4. Los tres estados del Circuit Breaker: **CLOSED** cuenta fallos, **OPEN** hace fail-fast con fallback, y **HALF-OPEN** prueba si el servicio se recuperó.

CUÁNDO SÍ • VENTAJAS

Por qué importa: mecanismo central de **FAULT TOLERANCE** — protege la disponibilidad del conjunto y mejora el MTTR. Se implementa a nivel **aplicación** (Resilience4j, Hystrix) o a nivel **service mesh** (sidecar, transparente). El diseño del **fallback** es tan importante como el breaker (source: circuit-breaker.md; Nygard, *Release It!*).

Service Mesh

DEFINICIÓN

Service Mesh: capa de infraestructura dedicada que gestiona la comunicación servicio-a-servicio **sin tocar el código**. Un **proxy sidecar** junto a cada instancia intercepta todo el tráfico (data plane); un control plane lo configura. Resuelve resiliencia (timeouts/retries/circuit breaking), seguridad (**mTLS** automático), observabilidad (métricas/tracing) y traffic management (canary, A/B), una vez y de forma uniforme. Ej.: Istio, Linkerd (source: service-mesh.md).

CUÁNDO NO · CUIDADO

Mesh — **cuándo no:** agrega **latencia** (cada hop pasa por dos proxies) y complejidad operativa. Con pocos servicios, hacer resiliencia/seguridad a nivel librería es más simple — la malla es over-engineering hasta cierta escala (source: service-mesh.md).

Orquestación vs Coreografía

DEFINICIÓN

Orquestación: un coordinador central (ESB en SOA, o servicio orquestador) **dirige** la secuencia $A \rightarrow B \rightarrow C$ — hay "director". **Coreografía:** sin director; cada servicio **reacciona a eventos** y publica los suyos — la lógica del flujo está distribuida (source: orquestacion-vs-coreografia.md).

TABLA 5. Orquestación vs Coreografía

Eje	Orquestación	Coreografía
Control del flujo	Centralizado, explícito	Distribuido, emergente
Visibilidad	Alta (en un lugar)	Baja (reconstruir de eventos)
Acoplamiento	Mayor (dependen del orquestador)	Menor (autónomos)
Punto único de falla	El orquestador	No hay uno central

TRADE-OFF

Control/visibilidad ↔ desacoplamiento/autonomía: flujos complejos con muchos pasos y necesidad de auditoría → orquestación; flujos simples donde la autonomía importa → coreografía. Es la misma decisión aplicada a la **Saga** (orchestration-based vs choreography-based). No hay ganador absoluto (source: orquestacion-vs-coreografia.md).

§4 Evolución del deployment: VM → Container → K8s → Serverless

Cómo se empaqueta y ejecuta un servicio fue **bajando de peso:** de *heavyweight*, *permanent*, *manual* (físico) a *lightweight*, *ephemeral*, *automated* (serverless) (source: evolucion-deployment.md).

TABLA 6. La línea de tiempo

Época	Unidad	Hito	Carácter
1990s	Máquina física	—	Máx. control, mín. densidad
2006	VM (service-per-VM)	AWS EC2	Imagen AMI con stack completo; aislamiento fuerte, arranque lento
2013	Container	Docker	Como VM pero lightweight: arranca en segundos, alta densidad, misma imagen dev→prod
2014	Serverless (FaaS)	AWS Lambda	Desaparece la máquina; solo se desarrollan servicios

DEFINICIÓN

Kubernetes: cuando hay muchos contenedores hace falta un **orquestador**. K8s da **resource management** (presenta una sola VM lógica), **scheduling** (dónde corre cada contenedor) y **service management** (mantiene la cantidad deseada de instancias, self-healing). Master (API Server, Scheduler, Controller Manager, etcd) + nodes (kubelet, kube-proxy, pods) (source: evolucion-deployment.md).

TRADE-OFF

Control ↔ **gestión administrada** · **aislamiento** ↔ **densidad** · **portabilidad** ↔ **lock-in**: no es "el más nuevo gana". Legacy/stateful con stack fijo → VM. Microservicios stateless a escala → Containers + Kubernetes (default actual). Cargas event-driven esporádicas sin estado → Serverless, **si se acepta el lock-in** y los límites de performance (no apto para procesos duraderos, trazabilidad difícil) (source: evolucion-deployment.md).

CUÁNDO NO · CUIDADO

K8s — **cuándo no**: su complejidad operativa lo vuelve over-engineering para pocos servicios; un PaaS gestionado o serverless puede evitar ese costo (source: evolucion-deployment.md).

§5 Tabla resumen: patrón → problema que resuelve

TABLA 7. Catálogo de Richardson de un vistazo

Patrón	Familia	Problema que resuelve
Descomposición	App/Data	Dónde cortar el sistema (by business capability / by subdomain)
Aggregate (DDD)	App/Data	Definir la frontera de consistencia transaccional dentro de un servicio
Database per Service	App/Data	Autonomía de datos por servicio (sin acoplar esquemas ni noisy neighbours)
Saga	App/Data	Transacción de negocio que cruza servicios sin 2PC (compensaciones)
CQRS	App/Data	Consultas cross-servicio sin JOIN (view-DB read-only por eventos)
Event Sourcing	App/Data	Historia/auditoría/reconstrucción temporal del estado
API Gateway	Communication	Punto de entrada único: ruteo, composición, cross-cutting de borde
BFF	Communication	API a medida por tipo de cliente (evita overfetching)
Circuit Breaker	Communication	Evitar cascading failure cortando llamadas a servicios caídos
Service Mesh	Communication	Resiliencia/seguridad/observabilidad uniformes sin tocar el código
Orquestación / Coreografía	Communication	Cómo coordinar la composición de servicios (control vs autonomía)
VM / Container / K8s / Serverless	Infrastructure	Cómo empaquetar y ejecutar cada servicio (control vs gestión administrada)

EN EL PARCIAL

Respuesta fuerte sobre microservicios: (1) ubicar el escalón en la **escalera evolutiva** y nombrar el **driver de negocio** (SOA = integración+gobernanza; micro = velocidad); (2) invocar el **triángulo de Conway** (sin equipos autónomos + CD → monolito distribuido); (3) por cada patrón citado, decir **qué problema resuelve y su costo** (consistencia eventual, latencia, complejidad); (4) cerrar con el **cuándo NO**: en org chica / dominio simple es over-engineering, un monolito hexagonal es mejor punto de partida (Brooks, "No Silver Bullet").